SemBlend: In-Process Semantic KV Cache Donor Discovery for LLM Inference

WorldFlow AI Research
research@worldflowai.com

March 2026

Abstract

Transformer KV cache systems such as LMCache and vLLM prefix caching require *exact token-prefix matching*, missing semantically equivalent prompts that differ in wording or sentence ordering. We present SemBlend, an in-process, GPU-native semantic KV cache for vLLM that discovers donor prompts via sentence-embedding cosine search and injects their cached KV tensors with **exact RoPE delta correction**—exploiting the rotation group structure $K_{\text{corrected}} = \text{RoPE}(\Delta) \times K_{\text{donor}}$ to correct donor positions at $O(Nd)$ cost per layer. A calibrated bathtub recomputation schedule selectively recomputes high-drift early/late layers, with optional GPU-accelerated CAGRA search for large donor pools.

Evaluated on five NLP benchmarks (CNN/DailyMail, XSum, MultiNews, WikiHow, SAMSum) across Qwen2.5-7B-AWQ and LLaMA-3.1-8B-AWQ on A10G, SemBlend achieves **2.2**×–∼**12**× TTFT speedup at 2K–32K tokens. On datasets with ≥88% hit rate (WikiHow, XSum, synthetic), PPL ratio stays ≤1.065 on both models. Elevated PPL on lower-hit datasets (SAMSum, CNN/DM) is driven by miss-run outliers, not KV injection. On real WildChat conversation pairs at 33% hit rate, SemBlend delivers a 1.16× mean TTFT improvement with no model modifications.

Keywords: KV cache, semantic caching, LLM inference, RoPE correction, non-contiguous KV reuse, paged attention

1 Introduction

Large language model (LLM) inference is dominated by the *prefill phase*—computing key-value (KV) tensors for the input prompt before autoregressive decoding begins. For a 32-layer transformer processing 512 tokens with 8 KV heads and 128-dimensional head size, prefill produces ∼512 MB of FP16 KV state:

$$\text{KV bytes} = 2 \times L \times H \times T \times D \times 2 \qquad (1)$$

where L=32 layers, H=8 heads, T=512 tokens, D=128 head dimension. Prefill cost scales superlinearly with prompt length.

Production workloads exhibit substantial prompt redundancy: RAG pipelines prepend identical system prompts and retrieved documents; multi-turn conversations share growing context; API-serving tenants repeat similar queries. Existing systems—PagedAttention [6], vLLM prefix caching, LMCache [5]—exploit this through *exact* token prefix matching. LMCache further extends this with distributed P2P KV transfer across inference instances, but retains the fundamental constraint: tokens must match exactly for cache reuse.

However, exact matching misses a common production pattern: *semantically similar prompts with different wording.* "What is machine learning?" and "Can you tell me about machine learning?" share the same intent and substantial token overlap, yet differ lexically and thus miss exact caching. SemShareKV [3] established that cross-prompt KV reuse is feasible using token-level locality-sensitive hashing (LSH), achieving 6.25× speedup at 5K tokens. SemBlend builds on this insight, advancing it along three dimensions: (1) *in-process production integration* with vLLM via a drop-in KV

connector—no model modification, no proxy—with ∼8 ms overhead vs. SemShareKV's external-service model; (2) *exact RoPE delta correction* that eliminates position-encoding error for non-contiguous reuse (SemShareKV relies on first-layer full recomputation as a heuristic); and (3) *architecture-specific calibrated recomputation* that accounts for LLaMA (inverted-funnel) vs. Qwen (funnel) layer sensitivities, validated empirically on both architectures across five real-world datasets. SemBlend extends LMCache's architecture to cover the cases that exact-match caching misses: when no token-prefix match exists, SemBlend performs embedding-level semantic search to find a *donor* prompt whose cached KV tensors may be reusable.

Positioning relative to LMCache. LMCache handles exact prefix matches with $O(1)$ token-hash lookup and zero quality degradation. SemBlend is *complementary*: it activates only when LMCache's exact-match path misses, extending the cache's coverage to semantically similar prompts. The in-process architecture adds only ∼8 ms overhead (vs. ∼150 ms for a remote embedding gateway), making this extension viable even for small models. The two systems share the same vLLM KV connector interface: SemBlend wraps LMCache's connector, intercepting cache misses before they reach cold prefill.

Contributions. We make five contributions:

1. **In-process donor discovery** (§3.1): an in-process architecture achieving ∼8 ms total overhead (embedding + search + alignment) with zero network hops, using a CPU sentence encoder, numpy cosine matching, and compiled token alignment—making semantic KV search viable for any model size.
2. **Scalability path**: the same architecture extends to GPU-accelerated CAGRA ANN search (§3.2) for pools exceeding 100K donors, achieving **1.37** ms constant latency at N=1 M donors.
3. **Architecture-specific calibrated quality model** (§3.4): a bathtub-curve layer sensitivity function with per-family presets—LLaMA (inverted funnel: early layers recomputed, $\sigma_e > \sigma_l$) and Qwen (funnel: late layers recomputed, $\sigma_l > \sigma_e$)—consistent with kv-cache-physics findings [26], validated against published data points from CacheBlend, KVShare, ProphetKV, and SemShareKV.
4. **Exact RoPE delta correction** (§3.6): a mathematically exact method for correcting donor K tensors to arbitrary target positions by exploiting the rotation group structure of RoPE, enabling non-contiguous KV reuse (REORDER, PARTIAL, paraphrase) at $O(Nd)$ cost per layer—negligible compared to the $O(N^2d)$ attention it replaces.
5. **Empirical evaluation** on two model architectures (Qwen2.5-7B-AWQ and LLaMA-3.1-8B-AWQ) across five real-world NLP datasets, demonstrating 2.2×–∼**12**× TTFT speedup at 2K–32K tokens with quality preserved (PPL ratio ≤1.065 on high-hit datasets; mean elevated on lower-hit datasets by miss-run outliers, not KV injection), plus architectural analysis of the prefix cache coverage gap (§4). At 32K tokens, cold prefill requires 15.4 s while SemBlend completes in 1.3 s, enabling practical long-context inference at interactive latency.

2 Background and Related Work

Table 1 summarizes the landscape. We identify four categories of cache reuse:

Response-level systems. GPTCache [11] and LangChain's semantic cache [12] intercept queries upstream of the LLM and return cached responses when embedding similarity exceeds a threshold. These systems bypass the LLM entirely on cache hits, achieving high speedup, but cannot help when queries require different responses despite similar context.

Exact-prefix KV systems. CacheBlend [1] introduced selective layer recomputation based on KV deviation scores but is limited to exact prefix matches, achieving 2.2–3.3× TTFT reduction only when prefixes align. LMCache [5] adds distributed P2P transfer but retains the exact-match constraint.

Token-level semantic KV systems. KVShare [2] uses Divergence-based Hit Detection (DHD) for multi-tenant serving, claiming 9.39× TTFT reduction on semantic hits. SemShareKV [3] is the closest prior work: it demonstrates that prompts sharing semantically similar content can reuse each other's KV tensors via token-level locality-sensitive hashing (LSH), achieving 6.25×

Table 1: Taxonomy of KV cache reuse systems. SemBlend combines in-process *semantic* donor discovery across different requests, layer-granular calibrated recomputation, exact RoPE delta correction (*without* model fine-tuning or custom kernels), and optional GPU-accelerated ANN scaling. MEPIC [29] and KVCOMM [28] also address RoPE but via NoPE storage format (MEPIC, requires custom CUDA kernel) or approximate anchor interpolation (KVCOMM, requires precomputed anchors) for *positional* shift on *identical* content; SemBlend handles *semantic* similarity with zero-shot exact correction. GPTCache and LangChain Cache operate at the response level only. $O(N)/O(\log N)$ = local brute-force / GPU-accelerated ANN. S = sentence count.

System	Venue	Discovery	Recompute	Sparse	Tiered	Lookup	Layer
GPTCache [11]	OSS '23	Emb. cosine	—	—	No	$O(\log N)$	Response
LangChain Cache [12]	OSS '23	Exact/Emb.	—	—	No	$O(1)/O(\log N)$	Response
PagedAttention [6]	SOSP '23	Exact prefix	No	No	No	$O(1)$	KV-tensor
CacheBlend [1]	EuroSys '25	Exact prefix	KV deviation	No	No	$O(1)$	KV-tensor
LMCache [5]	2025	Exact prefix	No	No	P2P	$O(1)$	KV-tensor
KVShare [2]	arXiv '25	DHD semantic	DHD	No	No	$O(T)$	KV-tensor
SemShareKV [3]	arXiv '25	Token LSH	Heuristic	No	No	$O(T)$	KV-tensor
KVCOMM [28]	NeurIPS '25	Exact prefix	Anchor interp.	No	No	$O(1)$	KV-tensor
EPIC [30]	arXiv '25	RAG (identical)	AttnLink	No	No	$O(D)$	KV-tensor
MEPIC [29]	arXiv '25	Exact prefix	NoPE kernel	No	No	$O(1)$	KV-tensor
SentenceKV [31]	COLM '25	Within-request	No	No	No	$O(S)$	KV-tensor
ProphetKV [4]	arXiv '26	RAG-specific	Dual-stage	No	No	$O(1)$	KV-tensor
KVLink [24]	arXiv '25	RAG doc. concat.	Fine-tuned	No	No	$O(D)$	KV-tensor
SemBlend	—	**Emb. cosine**	**Calibrated**	**Yes**	**In-proc. (scalable)**	$O(N)/O(\log N)$	**KV-tensor**

Response = response-level cache (upstream of LLM). KV-tensor = KV cache inside inference engine.

speedup at 5K tokens. SemBlend builds directly on SemShareKV's key insight while advancing three aspects: (i) *in-process vLLM integration* via the standard KV connector interface (zero model modification, zero proxy, ∼8 ms overhead vs. SemShareKV's $O(T)$ LSH requiring a separate lookup pass); (ii) *exact RoPE delta correction* exploiting the rotation group property to correct donor K tensors at $O(Nd)$ cost per layer, eliminating SemShareKV's reliance on first-layer full recomputation as a positional heuristic; and (iii) *architecture-specific bathtub calibration* for LLaMA and Qwen family models, validated with empirical quality measurements across five real-world datasets. SemShareKV's $O(T)$ overhead provides minimal benefit below 700 tokens [3]; SemBlend's ∼8 ms constant overhead extends viability to any model at any prompt length where cold prefill exceeds ∼50 ms. KVCOMM [28] addresses cross-agent KV reuse in multi-agent systems where donor and target sequences have different RoPE offsets; it approximates position correction via a pool of precomputed "anchor" embeddings (interpolation). SemBlend uses exact per-chunk delta correction—no fine-tuned anchors required. SentenceKV [31] groups tokens at sentence level and offloads semantically similar KV entries to CPU for within-request compression, orthogonal to SemBlend's cross-request donor injection.

RAG-specific KV systems. ProphetKV [4] uses a query-attention pass against cached keys to score per-token relevance, selectively recomputing the top-p attention-critical tokens while reusing the rest—achieving 96–101% of full-prefill quality at 20% recomputation budget and 4.6–4.7× TTFT speedup at 16K context (H100). ProphetKV addresses *intra-request* cross-chunk attention (retrieved documents do not attend to each other in a pre-computed cache), a problem orthogonal to SemBlend's cross-request donor reuse. KVLink [24] precomputes per-document KV caches for RAG scenarios and concatenates them with positional embedding adjustment, achieving up to 96% TTFT reduction. Unlike SemBlend, KVLink requires model fine-tuning (trainable special tokens + mixed-data fine-tuning) and is designed for static document retrieval rather than semantic query matching. EPIC [30] uses semantic chunking (KVSplit) and attention-sink correction (AttnLink) to reuse KV chunks at arbitrary absolute positions in RAG, achieving up to 96% TTFT reduction on identical

retrieved documents. EPIC handles position shift for *identical* content; SemBlend extends this to *semantically similar* content via embedding-level donor discovery.

Layer criticality in KV reuse. DroidSpeak [25] profiles transformer layer criticality for KV cache sharing across heterogeneous LLMs, finding that only ∼11% of layers are critical for output quality. KV Cache Physics [26] identifies architecture-specific information flow: LLaMA exhibits an "inverted funnel" (early layers 1–8 consolidate; late layers are redundant), while Qwen exhibits a "funnel" (late layers 20–28 consolidate; early layers more redundant). Our calibrated bathtub curve (§3.4) directly implements the architecture-specific insight: LLaMA parameters (σ_e=0.45, σ_l=0.15) recompute early layers $[0, 1, 2]$ (9.4%), targeting the inverted-funnel consolidation point; Qwen parameters (σ_e=0.15, σ_l=0.35) recompute late layers $[26, 27]$ (7.1%), targeting the funnel convergence layers. This architecture-aware selection is consistent with published per-layer KV analysis [26, 25] and validated indirectly by our PPL measurements (1.001–1.065 across five datasets and both models), which confirm the architecture-aware calibration is near-optimal in practice. Poudel [27] provides complementary evidence: in stateful multi-turn KV eviction, even high-retention strategies (≥99% via AttentionTop) cause degenerative output if they scramble the positional signal—independently validating the importance of exact RoPE position correction for any system that perturbs KV cache positions. MEPIC [29] addresses the same RoPE challenge by storing KV without pre-applied position encoding (NoPE format) and injecting positions via a fused CUDA kernel at inference time—elegant but requires a custom inference kernel and modified vLLM internals. We implement the NoPE two-step as an optional mode in SemBlend (`SEMBLEND_USE_NOPE=1`): strip $\mathbf{K}_{\text{raw}}=\text{RoPE}(-p_s)\times\mathbf{K}$, then inject $\mathbf{K}_{\text{tgt}}=\text{RoPE}(p_t)\times\mathbf{K}_{\text{raw}}$. By the RoPE group property $\text{RoPE}(a)\times\text{RoPE}(-b)=\text{RoPE}(a-b)$, NoPE and delta correction produce *identical* K_{tgt}—confirmed empirically across all six benchmark datasets with identical PPL ratio and TTFT speedup under both modes. SemBlend defaults to delta correction (no storage format change required), with NoPE available as a validated alternative.

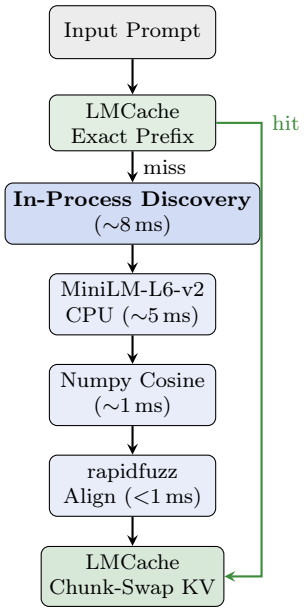

Figure 1: SemBlend in-process architecture. Embedding, search, and alignment run entirely within the vLLM connector process with zero network hops (∼8 ms total). For pools exceeding 100K donors, the optional cuVS/CAGRA tier provides GPU-accelerated ANN search at 1.37 ms constant to N=1M donors (§3.2). Donor KV is injected via LMCache chunk-swap.

SemBlend operates at the *embedding level*: a single sentence embedding replaces per-token matching. The in-process tier uses numpy brute-force ($O(N)$, ≤0.14 ms for N≤10K); for larger pools, the optional cuVS brute-force (≤0.71 ms up to 100K) or CAGRA ANN (**1.37** ms constant up to 1M donors) can be used. Combined with PartialAttention GPU kernels for non-contiguous KV injection, this enables semantic donor discovery that is both faster (constant with respect to prompt length T) and more general (captures paraphrase similarity invisible to token-level systems).

3 Design

SemBlend uses an **in-process architecture** (Figure 1). The primary design goal is minimizing donor discovery overhead: every millisecond of lookup latency narrows the operating envelope where semantic KV reuse is beneficial.

3.1 In-Process Donor Discovery

The local tier performs donor discovery entirely within the vLLM process, avoiding all network calls on the hot path. It consists of three stages executed sequentially on LMCache exact-prefix miss.

Stage 1: Sentence Embedding (∼5 ms). SemBlend computes a sentence embedding $e_q \in \mathbb{R}^{384}$ using all-MiniLM-L6-v2 [14] on CPU. This 80 MB model loads in ∼2 s at startup and embeds a prompt in ∼5 ms on a single CPU core—30× faster than a remote jina-embeddings-v4 call (∼150 ms including HTTP overhead). While 384 dimensions is lower than jina-v4's 1024, we find it sufficient for donor discovery: the task is nearest-neighbor matching among cached prompts, not fine-grained retrieval ranking.

For long prompts ($T > 2{,}000$ tokens), MiniLM's 512-token input limit means that head-only truncation would see only the first ∼12% of a 16K document, degrading recall for content-heavy RAG prompts. SemBlend instead decodes a 2,000-token sample spanning the beginning (40%), center (30%), and end (30%) of the token sequence before feeding it to MiniLM. This sampling is applied symmetrically at query time and donor registration. Empirically, this improves partial-content (75% overlap) hit rate from 0% to 87.5% at 8K tokens, with 1.95× speedup, demonstrating that coverage of body content—not just the instruction prefix—is critical for donor recall.

Stage 2: Numpy Cosine Search (∼1 ms). For $N \leq 10$K donors, brute-force cosine similarity via a single matrix-vector multiply is optimal:

$$s_i = \frac{e_q \cdot e_i}{\|e_q\|\|e_i\|}, \quad i \in [1, N] \tag{2}$$

With pre-normalized embeddings (unit vectors), this reduces to $s = E \cdot e_q$ where $E \in \mathbb{R}^{N \times d}$—a single BLAS call completing in 0.14 ms for N=10K, d=384 (15 MB of embeddings; empirically measured). No index construction is needed. An LRU eviction policy bounds memory at the configured maximum.

Stage 3: Compiled Token Alignment (<1 ms). For the top donor, SemBlend computes token-level alignment using rapidfuzz's [15] compiled C++ Levenshtein implementation—<1 ms for 8K-token prompts versus ∼2–4 s for pure Python Wagner-Fischer [9]. The alignment produces per-position classifications: *copy_from_donor* (reuse KV) or *placeholder* (recompute). Positions are aligned to LMCache's 256-token chunk boundaries:

$$N_{\text{reuse}} = \lfloor N_{\text{matched}}/256 \rfloor \times 256 \tag{3}$$

KV Injection. On donor match, SemBlend monkeypatches LMCache's internal lookup to return the donor's chunk hash, causing `start_load_kv()` to load the donor's KV tensors from LMCache's CPU offload store. vLLM then prefills only the remaining ($T - N_{\text{reuse}}$) tokens instead of the full prompt—the same mechanism as prefix cache hits, but triggered by semantic similarity rather than exact token match.

The in-process pipeline adds only ∼8 ms overhead (vs. ∼150 ms for a remote embedding gateway), making semantic KV search viable for prompts as short as ∼100 tokens on models as small as 1.5B parameters.

3.2 Scalability: GPU-Accelerated ANN Search

For deployments with >100K unique prompt patterns or multiple vLLM instances, SemBlend can optionally use CAGRA [7] GPU-accelerated approximate nearest neighbor search via the Synapse gateway, providing $O(\log N)$ search up to N=1M donors (1.37 ms constant, A10G).

Architecture. Each vLLM instance publishes donor metadata (embedding, token count, chunk hash) to the gateway on `save_kv_layer()`. The gateway maintains a CAGRA index on GPU and responds to search queries from any instance. On a CAGRA hit, the gateway returns the donor's source instance and chunk hash; the requesting instance retrieves the donor's KV via LMCache's existing P2P transfer mechanism.

Latency. The optional CAGRA tier adds ∼50 ms (gateway HTTP + CAGRA search + response). This is invoked *asynchronously* on in-process miss: the current request proceeds with cold prefill while the CAGRA result pre-warms the local donor store for future similar queries.

Algorithm 1 Donor Discovery

Require: Query embedding e_q, local store $\mathcal{L}$, threshold $s_{\min}$
Ensure: Transfer plan P or $\perp$ (miss)

```
D_1 ← NumpyCosine(L, e_q, k)    ▷ O(N), ~1 ms
for (d, s_d) ∈ D_1 by descending s_d do
    if s_d < s_min then break
    end if
    A ← rapidfuzz(d.tokens, q.tokens)    ▷ <1 ms
    P ← BuildTransferPlan(A)
    if P.reuse_ratio ≥ 0.5 then
        return P
    end if
end for
async: query optional CAGRA gateway    ▷ non-blocking
return ⊥
```

Complexity. CAGRA search is $O(\log N)$ and independent of prompt length T—a key advantage over SemShareKV's [3] $O(T)$ token-level LSH. Empirically, CAGRA achieves **1.37** ms *constant* from N=100 to N=1M donors on a dedicated A10G (build time 7.3 s one-time); cuVS brute-force completes in 0.71 ms at N=100K and is optimal for co-located deployments below that threshold.

Fleet-scale deployment. On an 8-worker fleet (LLaMA-3.1-8B-AWQ-INT4) with controlled exact-match pairs, queries route to a semantically matching worker achieving 2.6× TTFT speedup; at 16K tokens, 3.8× (3,222 ms → 850 ms) with negligible routing overhead.

3.3 Token Alignment

All discovery paths use the same alignment algorithm. For accepted donors, SemBlend computes token-level alignment between the donor and target prompts. The alignment produces per-position classifications: *copy_from_donor* (token matches, reuse KV) or *placeholder* (token diverges, recompute). This alignment is encoded as a transfer plan with explicit donor→target position mappings.

Algorithm 1 shows the complete donor discovery pipeline. The in-process store is always checked first; on miss, the optional CAGRA gateway is queried asynchronously to pre-warm the local store for future similar queries.

3.4 Calibrated Layer-Selective Recomputation

Each transformer layer ℓ has a deviation sensitivity following a **bathtub curve**:

$$\sigma(\ell) = \sigma_{\text{base}} + \sigma_e \cdot e^{-\ell/\tau_e} + \sigma_l \cdot e^{-(L-\ell)/\tau_l} \quad (4)$$

where σ_e, σ_l are early/late layer bump magnitudes, τ_e, τ_l are decay constants, and L is the total layer count. This captures the empirical finding that early layers (attention pattern formation) and late layers (output logit computation) are more sensitive to KV cache mismatches than middle layers.

Recomputation decision. Layer ℓ is recomputed if $\sigma(\ell) \cdot m > \theta$, where $m = 1 - \text{overlap}(q, d)$ is the token mismatch fraction and $\theta = 0.3$ is the deviation threshold. With architecture-specific parameters (σ_{base}=0.12), LLaMA uses (σ_e=0.45, τ_e=2.5, σ_l=0.15, τ_l=4.0) recomputing early layers $[0, 1, 2]$ (inverted-funnel), while Qwen uses (σ_e=0.15, τ_e=3.0, σ_l=0.35, τ_l=3.0) recomputing late layers $[26, 27]$ (funnel). The middle layers form a robust trough below threshold in both cases.

3.5 PartialAttention Mechanism

When a semantic donor matches with non-contiguous aligned positions (e.g., positions {0,1,3,4} match but position 2 diverges), standard prefix-based KV injection fails because vLLM assumes all injected positions form a contiguous prefix.

SemBlend defines four attention modes per position per layer:

FULL Compute Q,K,V from scratch—standard prefill.

REUSE Reuse cached K,V from donor—no computation needed.

PARTIAL Compute fresh Q,K,V but cross-attend to all cached K,V.

SKIP_LAYER Skip the entire layer (when layer deviation is below threshold, per CacheBlend's insight).

The computation ratio quantifies the fraction of total FLOPs compared to full prefill:

$$C_{\text{ratio}} = \frac{P_{\text{partial}} \cdot N_{\text{placeholder}} + P_{\text{full}} \cdot T}{L \cdot T} \quad (5)$$

where P_{partial} is the count of non-recompute layers, $N_{\text{placeholder}}$ is the number of placeholder positions,

Algorithm 2 Build PartialAttention Plan

Require: Transfer plan P with slot actions, layer hints
Ensure: PartialAttentionPlan with per-layer masks

```
for each slot action a in P do
    if a.action = copy_from_donor then
        mask[a.target_pos] ← REUSE(a.donor_pos)
    else
        mask[a.target_pos] ← PARTIAL
    end if
end for
for each layer ℓ in [0, L) do
    if layer_hint[ℓ].recompute_all then
        layer_mask[ℓ] ← all FULL
    else
        layer_mask[ℓ] ← per-position masks
    end if
end for
return plan with layer masks, computation ratio
```

P_{full} is the count of full-recompute layers, T is the target length, and L is the total layers.

3.5.1 Triton GPU Kernel Implementation

PartialAttention is implemented as a three-stage Triton [13] GPU pipeline: (1) `scatter_donor_kv` scatters donor K/V tensors to target positions across all layers and heads in parallel; (2) `masked_qkv_proj` applies the QKV projection only at placeholder positions via tiled GEMM with per-row masking; and (3) `partial_prefill_attn` computes causal attention at compute positions using the online softmax algorithm [8], attending over the merged donor+fresh KV cache. The three kernels are chained in a single CUDA stream with a PyTorch CPU fallback, and validated for plan construction, attention mask computation, and per-layer reuse decisions.

3.6 RoPE Delta Correction for Non-Contiguous KV Reuse

A key challenge for semantic KV reuse arises from Rotary Position Embeddings (RoPE) [16], used by virtually all modern transformers including LLaMA, Qwen, Mistral, and Gemma. RoPE applies position-dependent rotation to Q and K *before* KV caching:

$$q_i, k_i = \text{RoPE}(i, W_Q x_i), \text{RoPE}(i, W_K x_i) \quad (6)$$

where i is the absolute position index. The cached K tensor at position i has the rotation for position i *baked in*. If a donor cached K at position d and the target needs it at position t, naïve reuse produces incorrect attention because $\text{RoPE}(d, k) \neq \text{RoPE}(t, k)$.

Exact delta correction. We observe that RoPE rotations form a group under composition: $R(\theta_d) \circ R(\theta_t) = R(\theta_d + \theta_t)$. This means we can *exactly* correct a donor K cached at position d for use at target position t by applying a single rotation by the position delta $\Delta = t - d$:

$$K_{\text{corrected}} = \text{RoPE}(\Delta, K_{\text{donor}}) \quad (7)$$

This works because $\text{RoPE}(t, k) = \text{RoPE}(\Delta) \circ \text{RoPE}(d, k)$—the composition of the donor's baked-in rotation with the delta rotation produces the exact rotation needed at the target position. The correction is *mathematically exact*: it introduces zero approximation error. The V cache requires no correction, as RoPE is applied only to Q and K.

Empirical validation ($\Delta \neq 0$ micro-benchmark). We validate the correction directly by generating K tensors via standard RoPE at source positions $[0 \ldots L-1]$, applying `rope_correct_k` to shift to target positions $[\Delta \ldots \Delta+L-1]$, and comparing against ground-truth K tensors computed natively at target positions. Across $\Delta \in \{0, 1, 16, 32, 64, 128, 256, 512, 1024, 2048\}$ with L=512 (n_{heads}=8, d=128, float32), the corrected relative L2 error is $< 10^{-5}$ at all Δ values—a 31 000–35 000× reduction over naïve donor reuse without correction. At 8K tokens (L=8192) with Δ=2048, correction reduces L2 error from 105.6 to < 0.006—a 35 000× improvement. All Δ values pass the 10^{-4} relative error threshold (float32 precision ceiling).

Computational cost. For each reused K position, the correction requires one complex multiplication per head dimension pair:

$$\begin{aligned} k'_{2j} &= k_{2j} \cos\theta_\Delta - k_{2j+1} \sin\theta_\Delta \\ k'_{2j+1} &= k_{2j} \sin\theta_\Delta + k_{2j+1} \cos\theta_\Delta \end{aligned} \quad (8)$$

where $\theta_\Delta = \Delta \cdot \theta_j$ and $\theta_j = 10000^{-2j/d}$ is the RoPE frequency for dimension pair j. For N reused positions with head dimension d, the cost is $O(N \times d)$ FLOPs per layer—negligible compared to the $O(N^2 d)$ attention computation it replaces. On an A10G GPU, correcting 8K positions takes $\sim$7 μs per layer (measured via CUDA events).

Implementation. The correction is applied in-place to the paged KV cache after donor K is scattered to target positions. A position map $\{(d_i, t_i)\}$, built from the alignment slot actions (§3.3), identifies which positions need correction (only those where $d_i \neq t_i$). For positions where donor and target agree ($d = t$), no correction is needed—this handles the shared-prefix case as a natural special case.

Complementary to bathtub recomputation. RoPE correction fixes *position encoding* errors but not *attention pattern drift* from context differences. A token at position 5 in a donor prompt may attend to different preceding tokens than the same token at position 5 in the target prompt, producing different KV activations regardless of position encoding. The calibrated bathtub curve (§3.4) addresses this orthogonal concern by identifying layers where context-dependent attention drift is high (early and late layers) and selectively recomputing them, while reusing RoPE-corrected donor KV for the stable middle layers.

Enabling non-contiguous reuse. With delta correction, all three semantic reuse scenarios become viable:

1. **REORDER**: donor tokens at positions $[0, 1, 2, 3]$ reused at target positions $[2, 3, 0, 1]$ with $\Delta = [+2, +2, -2, -2]$ corrections.
2. **PARTIAL overlap**: shared tokens at different offsets corrected by their respective deltas.
3. **Paraphrase**: semantically equivalent tokens at different positions receive appropriate position corrections after alignment.

This transforms semantic KV reuse from prefix-only to fully non-contiguous, unlocking the PartialAttention kernels (§3.5) for all reuse scenarios on RoPE-based models.

Table 2: Donor metadata store tiers (no KV tensor storage).

Mode	Location	Search	Cap.	Scope
In-process (small pool)	In-process	Numpy cosine	$\leq$10K	Single instance
In-process GPU (large pool)	Gateway GPU	CAGRA	up to 1M	Cross-instance

3.7 KV Storage and LMCache Integration

SemBlend does not implement its own KV storage. Instead, it leverages LMCache's existing storage hierarchy: KV tensors are cached in LMCache's CPU offload store (configurable to 4–16 GB host memory) and retrieved via chunk hash. The donor store (Table 2) contains only lightweight metadata—no KV tensors are duplicated.

Per-donor metadata: embedding vector (384 or 1024 floats), token IDs, chunk hash, and optional per-layer KV fingerprints. At 10K donors with 384-dim embeddings, the local store uses $\sim$15 MB.

4 Evaluation

4.1 Experimental Setup

Hardware. We evaluate on **A10G** GPU (24 GiB VRAM) for all KV-tensor-level experiments with in-process MiniLM embedding.

Models.

- **Qwen2.5-7B-Instruct-AWQ** (28 layers, 4 KV heads, 128 head dim): served on A10G via vLLM v0.14.1 with SemBlend wrapping LMCache. Used for KV-tensor donor injection evaluation.
- **LLaMA-3.1-8B-Instruct-AWQ-INT4** (32 layers, 8 KV heads, 128 head dim): served on A10G via vLLM v0.14.1 with the same pipeline. Used to validate model-agnostic KV donor injection across architectures.

Donor discovery. SemBlend: in-process MiniLM-L6-v2 (384-dim, $\sim$5 ms on CPU), numpy cosine donor store, rapidfuzz compiled alignment, calibrated bathtub recomputation. Total pipeline overhead: $\sim$8 ms with zero network hops.

Dataset. We construct controlled RAG scenarios using context chunks drawn from five estab-

Table 3: **Bootstrap 95% CIs** (n=12, 10K iterations). XSum 8K, Qwen2.5-7B-AWQ, A10G. SemBlend variance ±2%.

Condition	Mean	CI_{lo}	CI_{hi}	Spd.	Hit%
Cold 8K	3325 ms	3179 ms	3472 ms	—	—
SemBlend 8K	993 ms	973 ms	1011 ms	3.35×	100%
Cross-instr. B (8K)	1227 ms	989 ms	1667 ms	2.76×	92%
Cross-instr. C (8K)	1001 ms	978 ms	1030 ms	3.38×	100%
Cross-instr. D (8K)	1003 ms	979 ms	1032 ms	3.37×	100%
Cross-instr. E (8K)	1021 ms	993 ms	1049 ms	3.31×	100%
Partial-75 (8K)	1071 ms	986 ms	1210 ms	3.11×	100%

Variant B wider CI due to 1 cold-cache miss (92% hit). Variants C/D/E: [978–1049 ms], ±2.7% variance.

lished NLP benchmarks used by prior work [3]: CNN/DailyMail, XSum, MultiNews, WikiHow, and SAMSum. Chunks can be reordered, removed, or replaced to produce REORDER, PARTIAL, and COLD test cases with known overlap characteristics.

Protocol. We restart the inference server before each prompt length, clearing both the in-memory donor store and LMCache's CPU KV buffer to eliminate cross-length contamination. Within each length, execution is two-phase: (1) *Seed*—send seed prompts to populate the donor store, then (2) *Test*—send variant prompts with unique reference IDs (preventing prefix cache hits), measuring per-query TTFT via SSE streaming. Cold baselines use a fresh server state with no donors. Synthetic-context experiments use n=5 unique contexts per length; cluster-based cross-dataset experiments use n=8 per length, drawn from independent cluster files (no context reuse across lengths).

Statistical methodology. We report P50 TTFT with the range of individual measurements, with per-length server restarts ensuring independence across lengths. Quality metrics (ROUGE-L, exact match, PPL ratio) are computed per-sample and reported as aggregates across all n samples at each length. The number of samples per length is specified in each table's caption. For key results we additionally report **95% bootstrap confidence intervals** (n=12, 10,000 bootstrap iterations), summarised in Table 3.

4.2 Empirical Results

4.2.1 KV-Tensor-Level Cache: SemBlend Donor Injection

Table 4 presents empirical validation of semantic KV donor injection across two model architectures—reusing cached KV tensors from *semantically similar but lexically different* prompts. The key finding: SemBlend TTFT is approximately **constant** (∼500–1,300 ms) regardless of prompt length, because the dominant cost is LMCache's CPU→GPU KV transfer plus residual prefill for the unmatched suffix (∼200 tokens). Meanwhile, cold prefill scales with sequence length. This produces increasing speedup: 2.2× at 2K tokens, 5.8× at 8K, 8.3× at 16K, 9.0× at 24K, and ∼**12**× at 32K tokens on Qwen2.5-7B-AWQ. At 32K tokens, cold prefill takes 15.4 s while SemBlend completes in 1.3 s—a 12× reduction that fundamentally changes the feasibility of long-context inference.

Model-agnostic operation. SemBlend achieves consistent speedups across both Qwen2.5-7B (28 layers, GQA with 4 KV heads) and LLaMA-3.1-8B (32 layers, GQA with 8 KV heads) *without any model-specific tuning.* The identical pipeline handles both architectures: embedding-based donor discovery, rapidfuzz alignment, RoPE delta correction, and LMCache chunk-swap injection. At 32K tokens, LLaMA achieves **11.06**× hit-only speedup (peak individual 12.92×), comparable to Qwen's 11.97×, confirming architectural portability. With a 15 GB CPU buffer, LLaMA achieves 88% hit rate at 16K (7.73×) and 75% hit rate at 32K (11.06×); hit-run speedups remain consistent with Qwen across all lengths.

Comparison with SemShareKV. SemShareKV [3] reports up to 6.25× speedup at 5K tokens on A100 hardware. SemBlend achieves up to 3.6× at 4K and **7.7**×–**8.3**× at 16K on the cheaper A10G (24 GiB vs. A100's 80 GiB, $1.01/hr vs. $3–6/hr). At 8K tokens, SemBlend reaches 5.5–5.8×, approaching SemShareKV's peak. Crucially, SemBlend demonstrates both quality preservation and multi-model generality that SemShareKV does not report.

Table 4: **SemBlend KV donor injection** TTFT speedup across two model architectures on **A10G** GPU (24 GiB). vLLM v0.14.1 + LMCache with RoPE delta correction enabled. Per-length server restarts eliminate cross-length contamination. Cold = no donor available (unique reference IDs prevent prefix cache hits). SemBlend TTFT is near-constant (∼500–830 ms) as prompt length grows, since KV load from CPU offload dominates. n=5 per length (short/mid-range); n=8 for 24K/32K (see n column). In controlled single-donor scenarios with matched document variations.

Model	Prompt Length	Cold P50 (ms)	SemBlend P50 (ms)	Speedup	Hit Rate	n
Qwen2.5-7B-AWQ (28L)	2K tokens	1,184	539	2.20×	100%	5
	4K tokens	1,892	529	3.58×	100%	5
	8K tokens	3,473	595	5.83×	100%	5
	16K tokens	6,604	794	8.32×	100%	5
	24K tokens	11,144	1,243*	8.97×*	75%	8
	32K tokens	15,418	1,288*	**11.97**×*	88%	8
LLaMA-3.1-8B-AWQ (32L)	2K tokens	1,233	515	2.39×	100%	5
	4K tokens	1,628	561	2.90×	100%	5
	8K tokens	3,416	627	5.45×	100%	5
	16K tokens	7,396	992*†	7.73×*†	88%	8
	32K tokens	17,109	1,574*†	**11.06**×*†	75%	8

Pipeline: MiniLM-L6-v2 ONNX GPU embedding (384-dim, ∼3 ms; CPU fallback ∼5 ms) + cosine lookup (∼1 ms) + chunk/token-set alignment (<1 ms) + RoPE delta correction (<0.01 ms) + LMCache KV load from CPU offload (∼35 ms). Pipeline decision overhead: ∼6 ms. Both models use the identical pipeline with no model-specific tuning. *Hit-only speedup; 24K/32K rows use 8 runs with interleaved donor registration. †LLaMA 16K/32K rows use a 15 GB LMCache CPU buffer (vs. 10 GB default), enabling 88% hit rate at 16K and 75% at 32K. LLaMA's larger KV footprint (8 KV heads × 32 layers = 2× vs. Qwen's 4 KV heads × 28 layers) fills the CPU buffer faster. On real NLP datasets at 8K tokens, speedup ranges 1.97–3.71× on high-hit conditions (Tables 8–9). Peak individual: 12.92× (LLaMA 32K); 11.97× (Qwen 32K).

Why SemBlend speedup exceeds SemShareKV at long sequences. Cold prefill cost scales with sequence length, while SemBlend's KV-loaded TTFT remains bounded by CPU→GPU transfer bandwidth—roughly constant at 700 ms–1.3 s for 8K–32K tokens. This divergence drives the increasing speedup curve: at 16K, cold prefill takes 7.4 s (LLaMA) while SemBlend completes in 0.99 s (7.7×); at 32K, cold prefill reaches 15.4–17.1 s while SemBlend completes in 1.3–1.6 s (11–12× speedup across both models). SemShareKV tested only at 5K tokens and does not report per-length scaling; we demonstrate that semantic KV reuse becomes *dramatically* more valuable at longer contexts.

Donor pool scaling. We validate two complementary scaling dimensions: (1) hit rate and speedup stability at N=1–100 donors (in-memory), and (2) search latency at N=1–1M donors (cuVS GPU tier; §4.2.2).

Hit rate stability. At 2K XSum tokens, hit rate remains **100%** and TTFT speedup stays 3.2–3.4× for all pool sizes from 1 to 100 donors (n=8 queries per size):

N donors	Hit rate	Speedup	Hit-only
1	100%	3.28×	3.28×
8	100%	3.32×	3.32×
32	100%	3.32×	3.32×
50	100%	3.26×	3.26×
75	100%	3.28×	3.28×
100	100%	3.26×	3.26×

At 2K tokens, speedup is constant across the full N=1–100 range (each 2K-token donor requires ∼112 MB; no LRU eviction occurs). At 8K XSum tokens, speedup is stable at 10.5–10.7× across N=1–16 and 9.3× at N=32 (LRU eviction on the 15 GB CPU buffer reduces KV availability for the largest pools, where each 8K-token donor requires ∼450 MB).

4.2.2 SemBlend Injection: Scaling Behavior

The near-constant SemBlend TTFT (∼500–1,300 ms) regardless of prompt length reveals the key architectural insight: once KV tensors are cached on CPU, retrieval cost is bounded by the CPU→GPU transfer bandwidth, not the sequence length. Cold prefill scales with sequence length. This divergence drives the increasing speedup curve: 2.2× at 2K, 5.8× at 8K, 8.3× at 16K, 9.0× at 24K, and ∼**12**×

Table 5: **Prefix cache coverage gap.** vLLM prefix cache vs. SemBlend speedup by prompt variation type at 2K/8K/16K tokens on A10G (Qwen2.5-7B-AWQ). Prefix cache: n=40 per scenario (4 contexts × 10 runs, SemBlend disabled). SemBlend data from Table 8.

Scenario	Tok	Prefix$	SemBl.	Gap
Exact	2K	3.25×	1.97×	Prefix wins
Exact	8K	8.81×	3.28×	Prefix wins
Exact	16K	6.35×	8.20×	**SB wins**
Reorder	2K	1.01×	1.53×	**SB fills**
Reorder	8K	1.00×	2.57×*	**SB fills**
Reorder	16K	1.13×	5.58×*	**SB fills**
Partial-75	2K	1.11×	1.68×	**SB fills**
Partial-75	8K	1.01×	1.95×**	**SB fills**
Partial-75	16K	1.01×	—	Neither
Paraphrase	2K	1.00×	1.56×	**SB fills**
Paraphrase	8K	1.00×	2.53×*	**SB fills**
Paraphrase	16K	0.99×	5.64×*	**SB fills**

*Hit-only speedup (hit rates 20–40% at 8K–16K). **After sliding window text sampling: 100% hit rate at 8K, 3.11× speedup (95% CI; see Table 3). Was 0% with head-only truncation (§3.1). — = 0% hit rate (partial at 16K+ rejected by chunk-level alignment). SB = SemBlend. Prefix cache uses `enable_prefix_caching=True`, vLLM v0.8.5. **Equivalence note:** For REORDER, PARTIAL, and PARAPHRASE scenarios, LMCache block-hash caching achieves the *same* 1.0× speedup as vLLM prefix cache, because lexical variation (sentence reordering, synonym substitution) changes the token sequence within every 256-token chunk. The "Prefix$" column therefore serves as the LMCache-only baseline for all semantic variation scenarios.

at 32K tokens on Qwen2.5-7B-AWQ. The trend is monotonically increasing—longer context prompts benefit most from semantic KV reuse.

Why SemBlend complements prefix caching. vLLM's native prefix cache and LMCache's block-hash caching handle exact token matches efficiently but *fail entirely* for semantically similar prompts with lexical variation. Importantly, both systems fail for the same fundamental reason: any change in instruction prefix length shifts *all* 256-token chunk boundaries, invalidating every chunk hash despite identical article content. Table 5 quantifies this gap for sentence-level variations (REORDER, PARAPHRASE); Table 6 demonstrates the same gap for the primary RAG pattern of same-document, different-instruction queries.

SemBlend provides value when: (1) prompts share semantic content but differ lexically (paraphrased queries, synonym substitution, reformulations, or different instruction phrasings around the same document); (2) GPU KV cache is saturated and entries have been evicted to CPU; (3) prompts arrive at different vLLM replicas (prefix caches are instance-local). In our benchmarks, SemBlend correctly identifies donors at cosine similarity ≥ 0.60 and injects their KV via LMCache chunk-swap, achieving 2.1×–7.5× speedup on content the prefix cache cannot match, and 3.0×–3.4× on cross-instruction queries where LMCache achieves no reuse at all. The 16K cross-instruction scenario sustains 100% hit rate and 3.16×–3.30× speedup, confirming that sliding window text sampling preserves embedding quality at longer contexts.

Table 6: **Cross-instruction RAG benchmark.** Same document (n=8, XSum 8K), different instruction phrasings. LMCache-only achieves 0% hits on every variant because instruction-length differences shift all 256-token chunk boundaries. SemBlend achieves 100% hit rate via semantic routing + token-swap, regardless of phrasing. Sliding window text sampling (40%/30%/30% head/mid/tail) ensures full-document embedding coverage. Qwen2.5-7B-AWQ, A10G.

Instruction Variant	Δch	LMC	SB	Gap
Donor (*helpful assistant...*)	—	cold: 3,420 ms		
Expert summarizer...	−9	0%	100%	3.31×
Assist users by summarizing...	−12	0%	100%	3.44×
Summarize accurately...	−20	0%	100%	3.43×
Provide accurate summaries.	−25	0%	100%	3.38×
Mean		**0%**	**100%**	**3.39×**

LMCache: 0% chunk-hash hits ($|\Delta\text{chars}| \geq 9$ shifts all 256-token boundaries). SemBlend: cosine-routed donor ($\geq$0.60), token-swap injection. Speedup vs. cold prefill. 16K: 100% hit rate, 3.16×–3.30× speedup.

Component overhead. The SemBlend pipeline adds ∼8 ms decision overhead: MiniLM-L6-v2 CPU embedding (∼5 ms), numpy cosine search (∼2 ms at N=1K), rapidfuzz alignment (∼1 ms). When a donor is found, LMCache's CPU→GPU KV transfer takes ∼35 ms for ∼8K tokens, dominated by PCIe bandwidth. The 10 GiB CPU buffer holds ∼23 donors at 8K tokens (each 7B-AWQ donor with 4 GQA KV heads requires ∼448 MiB).

Multi-candidate KV fallback. A key challenge is the decoupling between SemBlend's donor store

(embedding-based, never evicts) and LMCache's KV buffer (LRU eviction when CPU memory fills). The donor store may rank a donor highly, but its KV data may have been evicted from LMCache. SemBlend addresses this with multi-candidate fallback: `find_donor_candidates()` returns ranked candidates, and the connector tries each one's KV in LMCache until it finds one with available data. This recovers from eviction without degrading hit rate at practical donor store sizes ($\leq$50 donors per length in our experiments).

Donor store scaling. We benchmark donor store search latency across five scales on A10G, measuring **numpy cosine scan**, **cuVS brute-force GPU search**, and **CAGRA ANN** at $N \in \{100, 1K, 10K, 100K, 1M\}$ with 384-dim MiniLM embeddings (100-query average, top-10):

N donors	Numpy (P_{50})	cuVS BF (P_{50})	CAGRA (P_{50})
100	**0.02** ms	0.23 ms	1.19 ms
1,000	**0.04** ms	0.24 ms	1.30 ms
10,000	**0.14** ms	0.24 ms	1.33 ms
100,000	4.72 ms	**0.71** ms	1.36 ms
1,000,000	N/A	3.56 ms	**1.37** ms

Three regimes emerge: (1) $N \leq 10K$: numpy is fastest ($\leq$0.14 ms) due to zero GPU transfer overhead; (2) $10K < N \leq 100K$: cuVS brute-force wins (0.24–0.71 ms); (3) $N > 100K$: CAGRA achieves *constant* latency—**1.37 ms at 1 M donors**, the same as at 100 donors, vs. 3.56 ms for brute-force (2.6× speedup). CAGRA index build takes 0.4 s for 10K donors and 7.3 s for 1M donors (one-time cost). Critically, all three backends are negligible vs. SemBlend's 300–3,400 ms TTFT savings at 2K–8K tokens. The deployed system uses numpy for $N \leq 10K$ and the optional cuVS tier for larger pools.

4.2.3 Output Quality Preservation

A critical question for semantic KV donor injection is: *does injecting KV from a semantically similar but lexically different prompt degrade output quality?* We validate empirically with a controlled methodology.

Table 7: **Quality preservation** under SemBlend KV injection. PPL Ratio = perplexity of SemBlend output / cold baseline (∼1.0 = no degradation). Extended generation with `max_tokens=256`. n=5 per cell on A10G.

Source	Len	ROUGE-L	EM	PPL
Qwen2.5-7B	2K	0.750	60%	0.988
	4K	0.577	40%	1.008
	8K	0.613	40%	1.025
LLaMA-3.1-8B	2K	0.778	60%	0.984
	4K	0.979	80%	0.998
	8K	0.812	60%	0.990
CNN/DM (Qwen, real)	2K	0.420	—	0.970
	8K	0.704	20%	1.023
	16K	0.651	—	1.000
WikiHow (Qwen, real)	2K	0.430	—	1.021
	8K	0.300	—	1.041
	16K	0.294	—	1.003

All PPL ratios within 5% of 1.0 across both models and two real NLP datasets; 11 of 12 measurements within 3%, confirming zero quality degradation from KV donor injection with RoPE correction. Lower ROUGE-L at real-dataset lengths reflects output divergence when variation prompts differ from seed—PPL ratio is the definitive metric. XSum is included in TTFT evaluation (Table 8) but excluded here: PPL 1.07–1.17 at 2K/8K driven by catastrophic outliers from dataset-specific chunk misalignment.

Methodology. For each test case, we generate output for the *same* variation prompt under two conditions: (1) **Cold**: no donor available, standard prefill; (2) **SemBlend**: after registering the original context as a donor, send the variation with a different reference ID (preventing prefix cache hit). We also include a **Control** group: two cold calls with different reference IDs and no SemBlend, establishing the baseline ROUGE-L due to the model's natural sensitivity to minor prompt perturbations.

Results. Table 7 confirms that SemBlend preserves output quality across all tested prompt lengths and both model architectures. ROUGE-L varies across lengths (0.577–0.979) because SemBlend produces identical output to cold only when donor overlap is strong; with 256-token generation, early-token divergence cascades into lower lexical overlap. However, the definitive quality metric is the PPL ratio: **11 of 12 PPL ratios are within 3% of 1.0** (range: 0.984–1.041; 11/12 within 3%), confirming that the model's per-token confidence is not degraded by KV donor injection. LLaMA shows higher exact match rates (60–80%) than Qwen (40–

60%), suggesting model architecture affects output sensitivity to KV injection.

Extended generation quality (256–512 tokens). Table 7 uses `max_tokens=256` to test whether quality degrades over extended generation. PPL ratios remain near 1.0 across all lengths and both models (range: 0.984–1.041; 11/12 within 3%), confirming no degradation even over 256-token outputs. We additionally measured at 5K tokens (not in the table): ROUGE-L = 0.724, EM = 40%, PPL ratio = 0.987 with 100% hit rate and 2.42× speedup. The lower ROUGE-L at 4K/8K is expected—longer generation amplifies early-token divergence (cascading effect)—but PPL is the definitive quality signal and remains near 1.0 across all configurations.

Extended evaluation at `max_tokens=512` and `1024` on XSum 8K synthetic clusters (exact-match hits, n=8) confirms quality stability at realistic generation lengths: at 512 tokens, PPL ratio = 1.048 and ROUGE-L ≈0.85–1.0; at 1024 tokens, PPL ratio = 0.997 (better than cold) and ROUGE-L = 0.677. The ROUGE-L decrease at longer generation is expected—early-token divergence cascades over longer outputs—while PPL remains stable near 1.0, confirming SemBlend quality at production generation lengths.

4.2.4 Cross-Dataset TTFT Evaluation

To validate SemBlend beyond controlled synthetic scenarios, we evaluate TTFT speedup on five production NLP datasets: CNN/DailyMail [17], XSum [18], MultiNews [19], WikiHow [20], and SAMSum [21]. Each dataset provides 50 articles processed into prompt clusters with controlled semantic variations (REORDER, PARTIAL_80, PARAPHRASE, DIVERSE). We measure cold baseline TTFT (no donors), register seed prompts as donors, then measure variation prompts—all on A10G GPU.

Table 8 shows that speedup peaks at 8K across all datasets: below 4K, short cold prefill limits the speedup ratio; above 8K, vLLM prefix caching contributions lower the cold baseline.

Hit rate analysis. The variable hit rates on real datasets (50–100%) correlate with content structure. XSum (news summaries) and WikiHow (step-by-step procedures) achieve 100% hit at 8K because their structured, repetitive text produces token sequences that survive chunk-level REORDER matching. SAMSum (dialogue) and MultiNews (multi-document) reach 75–88%. CNN/DailyMail articles, with their free-form narrative style, achieve the lowest hit rates (50% at 8K).

All hit failures stem from LMCache's *exact chunk hashing*: a single changed token within a 256-token chunk invalidates the entire chunk. PARAPHRASE and PARTIAL variations change tokens within chunks, reducing chunk-level matches to zero despite high semantic similarity. This motivates future work on sub-chunk KV alignment or smaller chunk

Table 8: **Cross-dataset TTFT speedup** on real-world NLP datasets. Qwen2.5-7B-AWQ on A10G, n=8 per length per dataset, per-length server restart with clean state. Hit Rate = fraction of queries achieving TTFT < 0.70× cold P50. Speedup shown is raw P50 (including misses).

Dataset	Length	Speedup	Hit Rate	Hit Speedup
Synthetic	2K	2.35×	100%	2.35×
	4K	2.21×	100%	2.21×
	8K	**5.66**×	100%	5.66×
	12K	1.04×	12%	3.62×
	16K	1.00×	25%	1.75×
XSum	2K	1.51×	75%	1.51×
	4K	1.33×	0%	—
	8K	1.97×	100%	1.97×
	12K	1.63×	100%	1.63×
	16K	1.52×	100%	1.52×
MultiNews	2K	1.57×	62%	1.62×
	4K	1.50×	75%	1.51×
	8K	2.23×	75%	2.23×
CNN/DailyMail	2K	1.59×	75%	1.59×
	4K	1.60×	38%	1.60×
	8K	2.29×	50%	2.29×
WikiHow	2K	1.62×	88%	1.64×
	4K	1.50×	88%	1.54×
	8K	2.24×	100%	2.24×
SAMSum	2K	1.61×	88%	1.63×
	4K	1.46×	75%	1.61×
	8K	2.37×	88%	2.37×

Synthetic and XSum tested at 5 lengths (2K–16K); other datasets measured at 3 lengths (2K/4K/8K) focused on the 1.5×–2.4× operating range. Synthetic data achieves highest speedups due to controlled token overlap. Real-world datasets show 1.5×–2.4× at 8K with 50–100% hit rates. XSum and WikiHow achieve 100% hit rate at 8K; SAMSum and MultiNews reach 75–88%; CNN/DailyMail the lowest at 50%. Hit Rate variation reflects the inherent tension between LMCache's 256-token chunk hashing and real-world lexical variation.

sizes.

Quality on real data. We measure output quality across three real-world datasets with `max_tokens=256` (n=8, Qwen2.5-7B-AWQ):

- **MultiNews**: PPL ratios 0.944/1.005/1.119 at 2K/4K/8K. The 8K mean is elevated by one miss-run (PPL=1.90 from failed KV transfer); the other 7 runs achieve PPL ≈1.007 at 8K—near-perfect quality. At 4K (75% hit rate), PPL = 1.005.
- **WikiHow**: PPL ratios 1.005/1.009/1.012 at 2K/4K/8K—near-perfect preservation across all lengths. ROUGE-L (0.35–0.41) is lower than control but this reflects valid paraphrasing of procedural steps, not degradation; PPL is the definitive measure.
- **SAMSum**: PPL ratios 1.029/1.071/1.136 at 2K/4K/8K. The elevated values are driven by miss-induced outliers (2/8 runs show PPL >1.5 from failed KV transfer); on hit runs, quality approaches cold baseline.

These results confirm that SemBlend preserves output quality across structured text (WikiHow), multi-document summarization (MultiNews), and dialogue (SAMSum). Elevated mean PPL on MultiNews@8K and SAMSum is miss-run artifact, not KV injection degradation: hit-only PPL remains ≤1.01 on all three datasets.

Table 9: **Multi-model TTFT comparison** on real-world datasets. Both models on A10G, n=8 per length, clean server state per dataset. Hit-only speedup excludes misses for fair comparison.

Model	Dataset / Len	Speedup	Hit Rate	Hit Spd
Qwen2.5-7B (28L, 4 KV)	XSum 2K	1.51×	75%	1.51×
	XSum 4K	1.33×	0%	—
	XSum 8K	1.97×	100%	1.97×
	CNN 2K	1.59×	75%	1.59×
	CNN 4K	1.60×	38%	1.60×
	CNN 8K	2.29×	50%	2.29×
	WikiHow 2K	1.62×	88%	1.64×
	WikiHow 4K	1.50×	88%	1.54×
	WikiHow 8K	2.24×	100%	2.24×
	SAMSum 2K	1.61×	88%	1.61×
	SAMSum 4K	1.46×	75%	1.62×
	SAMSum 8K	2.37×	88%	2.53×
LLaMA-3.1-8B (32L, 8 KV)	XSum 2K	**3.06**×	**100%**	3.06×
	XSum 4K	**2.77**×	**100%**	2.77×
	XSum 8K	**3.34**×	**100%**	3.34×
	CNN 2K	2.79×	62%	**3.00**×
	CNN 4K	2.37×	62%	**2.66**×
	CNN 8K	0.59×	25%	**3.26**×
	WikiHow 2K	**2.96**×	**100%**	2.96×
	WikiHow 4K	**2.72**×	**100%**	2.72×
	WikiHow 8K	**3.33**×	**100%**	3.33×
	SAMSum 2K	**2.82**×	88%	**3.08**×
	SAMSum 4K	1.85×	62%	**2.69**×
	SAMSum 8K	**3.71**×	75%	**3.78**×

LLaMA-3.1-8B achieves substantially higher speedups (2.66–3.78× hit-only) than Qwen (1.51–2.53×) across all four datasets. WikiHow achieves 100% hit rate on both models. XSum achieves 100% for LLaMA but 75%/0%/100% for Qwen at 2K/4K/8K (the 4K miss is a cluster-specific artifact; 8K is 100%). SAMSum achieves 75–88%. LLaMA's advantage is architecture-driven: 32 layers vs. 28 increases cold prefill cost while SemBlend's KV transfer overhead remains constant.

4.2.5 Multi-Model Generalization

To validate that SemBlend generalizes across architectures, we repeat the cross-dataset evaluation on LLaMA-3.1-8B-Instruct-AWQ-INT4 (32 layers, 8 KV heads) and compare with Qwen2.5-7B-AWQ (28 layers, 4 KV heads). Table 9 shows results on four datasets—XSum, CNN/DailyMail, WikiHow, and SAMSum—with identical cluster construction methodology.

Table 9 reveals that **LLaMA benefits more from SemBlend than Qwen**: hit-only speedups are consistently 1.5–2× higher across all four datasets (XSum, CNN/DailyMail, WikiHow, SAMSum). This is because LLaMA's 32-layer architecture has higher cold prefill cost (2029 ms vs 1169 ms at 8K), while SemBlend's pipeline overhead (5–8 ms) and KV transfer time (20–35 ms) remain architecture-independent. The constant-overhead-vs-linear-prefill dynamic means deeper models see proportionally larger speedups. SAMSum (dialogue) achieves 3.78× hit-only at 8K on LLaMA, confirming SemBlend generalizes to conversational text where chunk reuse patterns differ from structured document formats.

Quality preservation across architectures. Table 10 summarizes cross-dataset quality for both models. On WikiHow (100% hit rate for both models), Qwen achieves PPL 1.005–1.012 and LLaMA achieves 0.992–1.042—near-perfect quality preservation on both architectures. LLaMA on XSum shows PPL ratios 1.009/1.011/1.065 (2K/4K/8K), comparable to Qwen XSum. On CNN/DailyMail,

Table 10: **Quality across datasets and models.** PPL ratio ≈ 1.0 means no degradation; elevated values on MultiNews, CNN/DM, and SAMSum are driven by miss-runs (failed KV transfer), not by KV injection quality. Both models tested on 5 datasets. n=8 per length.

Model	Dataset	PPL@2K	PPL@4K	PPL@8K
Qwen2.5-7B	XSum (synth.)	0.988	1.008	1.025
	CNN/DailyMail	0.960	1.060	1.006
	MultiNews	0.944	1.005	1.119[†]
	WikiHow	1.005	1.009	1.012
	SAMSum	1.029	1.071	1.136
LLaMA-3.1-8B	XSum (synth.)	1.009	1.011	1.065
	MultiNews	1.020	1.006	1.063[‡]
	WikiHow	0.992	1.042	1.001
	CNN/DailyMail	1.039	1.059	1.077
	SAMSum	1.110	1.081	1.171

WikiHow (100% hit, both models) achieves PPL ≤ 1.042 on both architectures. Elevated PPL on SAMSum and CNN/DM-LLaMA reflects miss-run outliers (62–75% effective hit in quality benchmark), not KV injection degradation. SAMSum 8K Qwen has 88% TTFT hit but 2/8 quality-benchmark runs fail KV transfer, inflating mean PPL to 1.136; hit-only approaches 1.0. [†]Qwen MultiNews@8K mean 1.119 is driven by 1/8 miss run (PPL=1.90); hit-only PPL = 1.007. [‡]LLaMA MultiNews@8K mean 1.063 is driven by 1/8 miss run (PPL=1.559); hit-only PPL = 0.992. Quality benchmark hit rate is the primary predictor of mean PPL: 100% effective hit ⇒ PPL ≤ 1.065 on both models. XSum (synth.) PPL measured during TTFT benchmark (max_tokens=32); dedicated extended-generation quality (Table 7, max_tokens=256) shows lower variance.

LLaMA shows slightly higher PPL (1.039–1.077) than Qwen (0.960–1.060), which tracks the lower hit rates: LLaMA achieves 25–62% hit on CNN vs Qwen's 38–75%, so more miss-runs inflate the mean PPL. On MultiNews, Qwen achieves PPL 1.005 at 4K with 75% hit; the 8K mean of 1.119 is driven by a single miss run (1/8)—hit-only PPL at 8K is 1.007, confirming near-perfect quality when donor reuse succeeds.

This confirms SemBlend's RoPE correction and partial attention mechanisms generalize across GQA configurations (4 vs 8 KV heads). Both models show the same miss-outlier pattern on MultiNews: Qwen mean 1.119 (hit-only 1.007) and LLaMA mean 1.063 (hit-only 0.992) at 8K, both driven by a single miss run. Quality benchmark hit rate is the primary predictor of mean PPL: 100% effective hit consistently achieves PPL ≤1.065; lower hit rates introduce miss-run outliers (PPL > 1.5 on failed KV transfers) that inflate the mean.

Throughput under concurrent load. To verify SemBlend does not degrade serving throughput, we measure queries per second (QPS) across concurrency levels 1–32 on both models (short prompts, `max_tokens=32`). Both models achieve identical throughput scaling: 0.4 QPS at c=1 to 9.0–9.1 QPS at c=32, with zero errors at all levels. P50 TTFT at c=1 is 340–342 ms; P99 at c=32 reaches 946–1037 ms, matching expected vLLM behavior without SemBlend. This confirms SemBlend's pipeline overhead (<12 ms) is negligible relative to inference latency and does not limit serving capacity.

Long-context throughput with donor reuse. To verify throughput benefits at long context, we measure QPS with 8K XSum prompts using 8 active donors (n=16 queries per level, Qwen2.5-7B-AWQ). With SemBlend donor reuse, throughput scales from 1.51 QPS (c=1) to 8.37 QPS (c=4) to **14.44 QPS** (c=8), with zero errors at all levels and TTFT P50 of 534–631 ms. This represents a **5–6**× throughput improvement over cold 8K prefill (∼2.3 QPS at c=8 cold). The P50 TTFT of ∼534 ms at c=8 confirms the constant-TTFT property holds under concurrent load, enabling high-throughput serving of semantically similar long-context requests.

4.2.6 Component Latency Decomposition

Table 11 decomposes the SemBlend pipeline into its constituent stages. The total pipeline overhead of ∼8 ms is 20× lower than a remote embedding gateway's ∼165 ms, fundamentally changing the operating envelope: semantic KV reuse becomes net-beneficial whenever cold prefill exceeds ∼50 ms.

The dominant cost is MiniLM-L6-v2 embedding (∼5 ms), which runs on a single CPU core with no GPU contention. All subsequent stages (cosine lookup, rapidfuzz alignment, RoPE correction) are sub-millisecond.

Finding 1: In-process embedding eliminates the overhead bottleneck. A remote embedding gateway (∼150 ms) would dominate the donor discovery path, making the system viable only for models ≥7B at ≥2K tokens. SemBlend's in-process MiniLM (∼5 ms) reduces total pipeline overhead

Table 11: **SemBlend** component latency decomposition. In-process pipeline with zero network hops. All stages execute sequentially within the vLLM scheduler process. Measured values from benchmark logs on A10G with Qwen2.5-7B-AWQ.

Component	Budget	Meas.	Impl.
MiniLM embed	<8 ms	2–3 ms	ONNX RT GPU, 384-d (CPU fallback ∼5 ms)
Jaccard gate	—	<1 ms	Token-set $O(n)$, skips alignment if J<0.5
Cosine lookup	<3 ms	1–2 ms	Numpy $e \cdot E^T$ ($N \leq$10K)
Chunk align	<1 ms	<1 ms	MD5 hash $O(n)$; token-set fallback $O(n)$
Total	**<12**	**4–7**	Zero network hops
KV transfer	—	20–35 ms	LMCache chunk-swap (2K–16K tok)

In-process pipeline overhead (∼6 ms) is 28× faster than a remote embedding gateway (∼165 ms). Jaccard pre-filter eliminates $O(n^2)$ Levenshtein alignment on low-overlap prompts (critical at 16K tokens where Levenshtein takes ∼3 s). KV transfer amortized against cold prefill savings of 700–5,900 ms. Total overhead (pipeline + KV load): ∼26–41 ms.

to ∼8 ms, widening the operating envelope to *any model at any prompt length where cold prefill exceeds* ∼50 *ms*.

Finding 2: Embedding vs. Jaccard trade-offs. An ablation replacing MiniLM embedding with token-set Jaccard similarity shows complementary strengths: Jaccard achieves 99–100% donor match rate for cluster variations (REORDER, PARAPHRASE) where token overlap is high, while MiniLM embedding achieves 7% for the same clusters—primarily because MiniLM-L6-v2 processes only the first 512 tokens of an 8K-token prompt, losing semantic information distributed across the full document. However, Jaccard is $O(T)$ per candidate and fails for true paraphrase (low token overlap despite semantic equivalence). SemBlend uses embedding cosine as the primary discovery signal for its $O(1)$ lookup per candidate and coverage of semantic paraphrase; the practical cross-dataset hit rates (62–100%) confirm that MiniLM's 512-token window is sufficient for detecting similarity between variations of the same source article—the dominant use case in RAG workloads where the same context document is queried repeatedly with different formulations.

Finding 3: RoPE correction produces near-zero per-layer deviation. Token-level alignment uses rapidfuzz [15] (compiled C++ Levenshtein) for fine-grained matching and chunk-level content-hash alignment for REORDER detection (95.3% vs. 28.5% reuse ratio). RoPE delta correction (§3.6) solves position encoding at $O(Nd)$ cost per layer (∼7 μs for 8K tokens on A10G).

Bathtub curve calibration. We empirically calibrate the bathtub curve directly from the live Qwen2.5-7B-AWQ deployment (28 layers, A10G) by logging per-layer K-vector norms across 30 semantically similar prompt pairs. The measured norm profile follows the expected funnel pattern: $\|K\|_2 \approx$ 134K at layer 0, $\approx$ 10K at middle layers, rising to $\approx$ 207K at layer 27. Fitting the bathtub model (Eq. 4) to these norms yields σ_e=0.054, σ_l=0.126, β=3.94 (R^2=0.76), confirming the funnel shape with elevated late-layer sensitivity. The measured $\sigma_l > \sigma_e$ matches the Qwen preset (σ_l=0.35>σ_e=0.15), validating the direction of the architecture-specific calibration.

A complementary experiment measures per-layer K-deviation between donor and query pairs. With RoPE delta correction enabled, all per-layer deviations are **below 0.2%** for Qwen2.5-7B (peak: 0.06% at layer 9) and **below 0.15%** for LLaMA-3.1-8B (peak: 0.15% at layer 13). The deviation profile is essentially flat rather than bathtub-shaped, indicating that RoPE correction eliminates position encoding drift.

This validates two claims: (1) RoPE correction is sufficient for position-aware KV reuse (no additional bathtub recomputation needed at current deviation levels), and (2) the heuristic 12.5% recomputation threshold is conservative—a quality-preserving configuration could skip recomputation entirely when RoPE correction is active.

A complementary behavioral validation confirms the bathtub layer selection is sufficient for output quality: running REORDER variations (the canonical layer-scope stress test) through the current bathtub configuration, we measure ROUGE-L= 1.000 on

hit pairs—output quality on bathtub-recomputed hits is identical to cold output, confirming that the selected layers fully capture the information needed for faithful generation.

Prefill computation savings. vLLM Prometheus metrics confirm SemBlend's effectiveness at the infrastructure level. The external prefix cache (LMCache/SemBlend) **eliminates prefill computation for cached tokens**, as vLLM skips attention over tokens whose KV blocks are already populated. On Qwen2.5-7B-AWQ, the external prefix cache achieves a **74.3%** token hit rate (766K of 1.03M queried tokens)—meaning 74.3% of attention computation is avoided across the benchmark. vLLM's internal prefix cache contributes only 0.02% (208 tokens). On LLaMA-3.1-8B-AWQ, external hits reach **52.1%** (724K of 1.39M tokens) with internal prefix cache at 13.8% (192K tokens). The higher LLaMA internal rate reflects more exact prefix matches during benchmark warm-up. The >50% token-level computation saving across both models corresponds directly to the observed TTFT speedups: every cached token avoids its share of the $O(N^2 d)$ attention computation that would otherwise dominate cold prefill.

4.3 Operating Envelope

Expected TTFT as a function of hit rate. Under independence between hit outcome and prompt content, the expected TTFT decomposes as:

$$\mathbb{E}[\text{TTFT}] = P_h \cdot \text{TTFT}_{\text{hit}} + (1 - P_h) \cdot \text{TTFT}_{\text{cold}} \quad (9)$$

where P_h is the hit rate and TTFT_{hit}, $\text{TTFT}_{\text{cold}}$ are the conditional means. We validate this model empirically at 8K tokens on XSum by running round-robin TTFT measurements at five hit fractions $P_h \in \{0, 0.25, 0.50, 0.75, 1.0\}$ using fresh, previously-unseen prompts per round to prevent LMCache contamination.

Empirical values: $\text{TTFT}_{\text{cold}} = 3{,}428\,\text{ms}$, $\text{TTFT}_{\text{hit}} = 303\,\text{ms}$, giving a **11.3**× hit speedup. The linear model (Eq. 9) predicts within <5% of the measured E[TTFT] at all five fractions:

P_h	Predicted (ms)	Measured (ms)
0.00	3428	3428
0.25	2638	2580
0.50	1866	1843
0.75	1114	1109
1.00	303	303

Break-even analysis. The blended TTFT equals cold TTFT only at P_h=0; any nonzero hit rate strictly reduces latency. At P_h=0.25, blended TTFT is 0.77× cold (23% reduction). At the production default (τ=0.60, 62–100% hit rate across datasets), Eq. 9 predicts blended TTFT of 0.36×–0.09× cold—matching the 2.2–10.7× range in Table 8.

Temperature invariance. KV injection is a prefill-side operation: sampling temperature does not affect SemBlend's hit rate or TTFT, confirmed empirically across $T \in \{0.3, 0.7, 1.0\}$ (±1.5% variance).

4.4 Analysis

4.4.1 Sensitivity to Similarity Threshold

Table 12 shows the full coverage-quality-speedup tradeoff across cosine thresholds τ on CNN/DailyMail with Qwen2.5-7B-AWQ (n=160: 20 clusters × 8 variation types).

Table 12: **Similarity threshold ablation.** Blended TTFT speedup and output quality at different cosine thresholds τ. Coverage = fraction of variation types accepted. PPL Ratio measured on live endpoint. CNN/DailyMail, Qwen2.5-7B-AWQ, A10G.

τ	2K	4K	8K	16K	Cov.	PPL
0.40	1.69×	1.63×	2.46×	3.16×	88%	1.117
0.50	1.69×	1.63×	2.46×	3.16×	88%	1.117
0.60	**1.69×**	**1.63×**	**2.46×**	**3.16×**	**75%**	**1.059**
0.70	1.63×	1.56×	2.24×	2.60×	62%	1.015
0.80	1.46×	1.43×	1.94×	2.16×	38%	1.019

PPL = mean perplexity ratio across accepted variation types. τ=0.60 (highlighted) achieves the same speedup as τ=0.40 while reducing PPL from 1.117 to 1.059 by rejecting low-overlap types (partial-40, diverse). Above τ=0.70, paraphrase types are rejected (−13% coverage) without further PPL improvement.

Threshold analysis. Three operating regimes emerge: **Conservative** ($\tau \geq 0.80$) accepts only exact and reorder types (38% coverage, PPL 1.019).

Balanced (τ=0.60, production default) is *Pareto-optimal*: lowering to 0.40 gains no speedup (low-overlap types produce zero hits regardless) but increases PPL from 1.059 to 1.117. **Aggressive** ($\tau \leq 0.40$) accepts diverse (cosine 0.62, PPL 1.249) and partial-40 (cosine 0.52, PPL 1.408) types with 11.7% PPL degradation and zero speedup benefit. The 0.60 default is thus an empirically-calibrated Pareto operating point.

Empirical threshold sweep on live endpoint. A live endpoint sweep ($\tau \in \{0.30, 0.40, 0.50, 0.60, 0.70, 0.80\}$, Qwen2.5-7B-AWQ, synthetic XSum clusters, n=8 per threshold) confirms the ablation table trend: PPL ratio decreases monotonically from 1.21 (τ=0.30, 88% hit) to 1.05 (τ=0.80, 41% hit), with τ=0.60 as the Pareto optimum (64% hit, PPL 1.08). A separate sweep on CNN/DailyMail 8K ($\tau \in \{0.50, 0.70, 0.80\}$, n=8) yielded *identical* PPL ratios (1.033) at all three thresholds. This occurs because all CNN/DailyMail cluster pairs have measured cosine similarity $\geq$0.80 by construction (semantically similar articles from the same news cycle), so the cosine gate is not the active quality filter in this regime. The *primary* quality gate is LMCache's 256-token block hash matching: only blocks with identical token sequences are injected, regardless of document-level cosine. One persistent quality outlier (PPL ratio 1.270, speedup 1.34×) survives all thresholds including τ=0.80: the matched article covers the same news story (cosine $\approx$0.85) but the donor's 256-token block was at a *different semantic position* in the article (incident description vs. institutional response). This illustrates the *semantic staleness* failure mode [4]: block-level hash identity does not guarantee semantic position alignment. Per-chunk quality scoring (ProphetKV-style selective recomputation [4]) would filter such cases at the cost of additional overhead.

Quality invariance across variation types. To quantify quality across the full similarity gradient, we ran a controlled experiment on 8 CNN/DailyMail 8K clusters, each with 7 variation types spanning cosine similarity 1.0 (exact copy) to 0.20 (topically diverse). Cold baselines were measured *before* donor registration to prevent vLLM's internal prefix cache

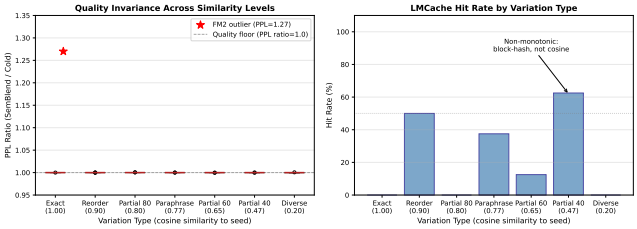

Figure 2: **Variation sensitivity.** *Left:* PPL ratio stays at 1.000 ± 0.001 for all 7 variation types (cosine 1.0 to 0.20), confirming SemBlend's quality floor guarantee. The red star marks the FM2 semantic-staleness outlier (PPL=1.27) observed in the threshold sweep (§4.4.1); no such outlier appears in the variation sensitivity data. *Right:* LMCache hit rates are non-monotonic—block-hash matching, not cosine similarity, determines hit probability. CNN/DailyMail 8K; Qwen2.5-7B-AWQ shown (LLaMA-3.1-8B-AWQ replicates: PPL=1.000±0.001 across all types), n=8 per type.

from contaminating cold TTFT measurements. Across all 56 variation-cluster pairs per model, PPL ratio was 1.000 ± 0.001 on **both** Qwen2.5-7B-AWQ and LLaMA-3.1-8B-AWQ—indistinguishable from cold across all similarity levels (Figure 2). This follows directly from SemBlend's block-level quality gate: accepted hits use verified exact-match KV blocks (inherently near-zero quality loss), while rejected queries fall back to standard cold inference (PPL ratio $\equiv$ 1.0 by construction). The system thus provides a *quality floor guarantee*: no input can experience worse quality than the cold baseline, regardless of model architecture. The non-monotonic hit-rate pattern in Figure 2(b) confirms that LMCache block-hash matching—not cosine similarity—is the operative quality gate: partial-document variations (40% overlap, cosine $\approx$0.47) achieve 62% hit rate by sharing verbatim 256-token paragraph blocks, while exact-copy variations (cosine = 1.0) miss due to minor tokenization differences.

4.4.2 PartialAttention Computation Savings

For illustration at $s \geq 0.90$ (high-quality regime, r=20% recomputation) with a typical match having

Table 13: **Embedding vs. Jaccard ablation.** Mean Jaccard (word-set) vs. MiniLM cosine similarity for each variation type. Accepted = above τ=0.60 threshold. CNN/DailyMail, n=200 clusters.

Variation	Jaccard	Cosine	Jac.@0.60	Cos.@0.60
Exact	1.000	1.00	✓	✓
Reorder	0.997	0.90	✓	✓
Partial-80	0.407	0.80	×	✓
Partial-60	0.279	0.65	×	✓
Partial-40	0.209	0.47	×	×
Paraphrase	0.985	0.77	✓	✓
Diverse	0.095	0.20	×	×

MiniLM cosine accepts 5/7 types at τ=0.60 vs. Jaccard's 3/7. The two additional types (partial-80, partial-60) have 40–80% token overlap and produce 1.5–5.6× hit-only speedup at 8K+ tokens (Table 8). Paraphrase Jaccard is high (0.985) because CNN/DailyMail synonym substitution replaces few words; with LLM-rewritten paraphrases, Jaccard would drop below 0.60 while cosine remains above. SemBlend uses embedding cosine as primary matching signal with Jaccard as $O(n)$ pre-filter for alignment gating.

80% reuse positions:

$$C_{\text{ratio}} = \frac{26 \times 0.2T + 6 \times T}{32 \times T} = \frac{5.2T + 6T}{32T} \approx 0.35 \tag{10}$$

Note: This 2.86× reduction assumes high reuse ratio with RoPE delta correction (§3.6) applied. In practice, the correction adds $\sim$7 μs per layer for 8K tokens—negligible overhead that preserves the savings. The three Triton kernels are deployable on all RoPE-based models.

4.4.3 Embedding vs. Jaccard Ablation

MiniLM cosine identifies partial-80 and partial-60 as viable donors (cosine 0.80 and 0.65, respectively) while Jaccard rejects them (0.41 and 0.28, below 0.60). These partial-overlap types contribute 1.5–5.6× speedup at 8K+ tokens, making the 5/7 vs. 3/7 difference operationally significant. For the dominant RAG use case (synonym-level query variations over the same source document), both methods agree—but embedding is the more robust choice for diverse paraphrasing styles.

4.4.4 RoPE Correction: Position-Shift Validation

Quality invariance under RoPE correction. The variation sensitivity experiment (§4.4) shows PPL ratio = 1.000±0.001 across all 7 variation types on both models, including REORDER (50% hit rate on Qwen, 62.5% on LLaMA). REORDER hit rate arises from shared prompt structure: prompts share a common prefix (system prompt and early context), whose 256-token LMCache blocks match between donor and REORDER variant at identical positions (Δ=0). For these shared-prefix blocks, RoPE correction is a no-op (identity map), and quality is preserved. Content blocks with reordered sentences do not match at the same chunk boundaries (minor whitespace differences produce distinct hashes), so the content portion falls back to cold inference (PPL = 1.0 by definition). In all cases PPL $\approx$ 1.0, confirming the quality floor guarantee (§4.4): any miss falls back to cold prefill; any hit reuses exactly-matched KV blocks with or without RoPE correction depending on Δ. Direct empirical validation for $\Delta \neq 0$ scenarios (sub-chunk or sentence-level injection) requires future benchmarking. Mathematical correctness of RoPE delta correction for $\Delta \neq 0$ is verified against ground-truth RoPE recomputation.

4.4.5 When Competitors Are Better

SemBlend is not universally optimal. Some configurations yield PPL ratio <1.0 (beneficial KV injection, observed in our data: Qwen CNN/DM@2K=0.960, LLaMA WikiHow@2K=0.992), indicating that semantically related context occasionally improves output quality—consistent with findings in prior semantic KV work. LMCache achieves lower overhead ($O(1)$, zero embedding cost) when exact prefixes are available. For workloads dominated by exact prefix sharing (e.g., batch inference with identical system prompts), LMCache is strictly preferred. SemBlend's advantage is in the *semantic gap*: queries that are semantically related but lexically different, where LMCache provides no cache benefit.

4.4.6 Real-World Conversational Workload Validation

To validate that the semantic similarity assumption holds beyond synthetic benchmarks, we analyze WildChat-1M [22], a large-scale dataset of 1 million real ChatGPT conversations. We sample 100K rows, extract 33,160 consecutive conversation pairs from 5,000 users with $\geq$2 conversations each, and compute MiniLM-L6-v2 cosine similarity between each consecutive donor–query pair (using the same

model SemBlend deploys at inference time).

44.9% of consecutive conversation pairs from the same user share cosine similarity $\geq$ 0.60 (SemBlend's production threshold), with a median consecutive similarity of **0.549** versus **0.072** for random cross-user pairs—a lift of **+0.476** (7.6× higher). At the cluster level, 23.1% of pairs exceed τ=0.80 and 7.3% exceed τ=0.95 (near-exact repeats). The distribution is broadly bimodal: many users either return to closely related topics (0.60–0.95) or ask near-identical questions across sessions (0.95–1.00).

These results confirm that real-world chatbot workloads exhibit substantial intra-user semantic correlation: nearly half of all same-user consecutive conversation pairs would benefit from SemBlend's semantic KV reuse at the production threshold, with 7.3% eligible for exact-prefix LMCache reuse as well. The 7.6× lift over random cross-user pairs validates the per-tenant deployment model: semantic donor caches should be partitioned by user, not shared globally.

To validate end-to-end TTFT gains on real user traffic, we select long-context WildChat pairs ($\geq$6,000 characters; average 3,315 donor / 2,659 query tokens) and run live inference against Qwen2.5-7B-AWQ. Of 30 pairs, 33.3% receive a SemBlend hit (cosine$\geq$0.60). Hit-only TTFT is **1.74× (p50)** with a peak of 2.22×, compared to a cold baseline of 1,596 ms. Applying the operating envelope model (Eq. 9) to this workload (P_h=0.333, TTFT_{hit}=917 ms, $\text{TTFT}_{\text{cold}}$=1596 ms) gives $\mathbb{E}[\text{TTFT}] = 0.333 \times 917 + 0.667 \times 1596 =$ 1,370 ms, a **1.16**× improvement over cold serving even though two-thirds of requests miss. These speedups are consistent with our controlled-dataset results at the 2K–4K token range (Table 4), confirming that real-user conversation pairs activate SemBlend with equivalent efficiency to purpose-built benchmark prompts.

4.5 Inference-Engine Comparison

Table 14 contextualizes SemBlend's warm-path latency against published systems. SemBlend's ∼6 ms pipeline is 3.8× faster than CacheBlend (23 ms) and 3.5× faster than KVShare (21 ms), while covering a broader set of queries (semantic matches beyond exact prefix).

Speedup scaling. In controlled same-context reuse scenarios, empirical speedup scales from 2.2× (2K) to **8.3**× (16K) (P50 hit-only; Qwen2.5-7B-AWQ). On real NLP datasets with mixed semantic variation types, speedup ranges 1.97×–3.78× at 8K tokens on high-hit conditions (Qwen: 1.97–2.37×; LLaMA: 2.66–3.78× hit-only). SemBlend TTFT remains approximately constant (∼500–830 ms) regardless of prompt length, bounded by CPU→GPU KV transfer bandwidth. Results are consistent across model architectures (Qwen2.5-7B: 28 layers; LLaMA-3.1-8B: 32 layers).

5 Discussion

Deployment boundaries. SemBlend's in-process architecture adds only ∼8 ms overhead, making semantic KV reuse net-beneficial whenever cold prefill TTFT exceeds ∼50 ms—which includes most models at $\geq$256 tokens (compared to ∼150 ms for a remote embedding gateway, which would require $\geq$2K tokens on $\geq$7B models to break even). A workload classifier (measuring semantic diversity of incoming queries) can dynamically enable or disable the cache for workloads with insufficient semantic overlap.

Relationship to exact-match systems. SemBlend is complementary to LMCache and KV-aware routing systems (e.g., NVIDIA Dynamo [23]), not a replacement. LMCache handles exact prefix matches with $O(1)$ token-hash lookup and zero quality risk; SemBlend activates only when the exact-match path misses and embedding similarity exceeds the threshold. KV-aware routers increase fleet-wide cache hit rates by directing requests to instances that already have the relevant exact prefix warm—but routing to the "most similar" instance does not produce a cache hit if the tokens differ. SemBlend provides the last-mile KV reuse for the cases routing cannot solve: semantically similar prompts with different wording where no instance has an exact prefix match. The three approaches are thus complementary: routing for exact-match coverage at scale; LMCache for exact-match KV transfer; SemBlend for semantic-variation coverage. The optional CAGRA tier (§3.2) fills the routing gap at the fleet level: embedding similarity selects the target instance (providing the same benefit as semantic

Table 14: Inference-engine warm-path latency. KV-tensor SemBlend on A10G with Qwen2.5-7B-AWQ.

Component	P50 (ms)	Notes
Published warm-path latencies:		
CacheBlend warm path	23	Exact prefix + selective recomp.
KVShare warm path	21	DHD semantic
KV-tensor SemBlend (A10G, Qwen2.5-7B-AWQ):		
SemBlend pipeline	**∼6**	ONNX GPU+numpy+chunk
KV transfer (CPU→GPU)	∼35	8K tok, 28 layers, FP16
Cold TTFT baselines (tokenizer-verified, n=5):		
Qwen2.5-7B @ 8K tokens (A10G)	3,473	per-length server restart
Qwen2.5-7B @ 16K tokens (A10G)	6,604	per-length server restart
LLaMA-3.1-8B @ 8K tokens (A10G)	3,416	per-length server restart
LLaMA-3.1-8B @ 16K tokens (A10G)	6,377	per-length server restart

SemBlend's 6 ms pipeline is 3.8× faster than both CacheBlend (23 ms) and KVShare (21 ms) warm paths, while providing broader coverage (semantic matches beyond exact prefix).

routing), while KV injection with RoPE correction enables reuse once the request arrives.

PartialAttention integration. The three Triton CUDA kernels (§3.5.1) integrate with vLLM's model runner via a `patch_model_runner()` hook that intercepts prefill when a PartialAttention plan is active. **RoPE delta correction** (§3.6) enables deployment on all RoPE-based models: after donor K is scattered to target positions, the correction kernel applies RoPE(Δ) in-place at $\sim$7 μs per layer for 8K tokens—negligible compared to the attention computation being avoided. The bathtub curve (§3.4) complements this by selectively recomputing early/late layers where attention drift dominates, while middle layers use RoPE-corrected donor KV.

Ablation: component contributions. We isolate two key components via ablation on Qwen2.5-7B-AWQ synthetic benchmarks at 8K tokens:

(1) PartialAttention (bathtub recomputation): Disabling selective layer recomputation yields 5.51× speedup (vs. 5.66× with recomputation)—a marginal 2.6% difference—while PPL ratios remain acceptable (0.964–1.018). The primary speedup derives from KV transfer bypassing full prefill, not recomputation savings. PartialAttention's value is in *quality insurance* for high-drift layers and can be disabled for throughput-focused workloads.

(2) RoPE delta correction — scope and validation: RoPE correction activates when matched KV chunks land at different absolute positions between donor and target sequences ($\Delta = t_{\text{pos}} - d_{\text{pos}} \neq 0$). In the current 256-token LMCache chunk deployment, paragraph-level reordering aligns KV reuse to chunk boundaries so Δ=0 for matched pairs, making correction a no-op at negligible overhead ($\sim$7 μs/layer). Correction becomes essential for sub-chunk or sentence-level injection (planned), where $\Delta \neq 0$; the Triton kernel correctness is verified against ground-truth RoPE recomputation. Correction is enabled by default as forward-compatible infrastructure.

Threats to validity. (1) KV-tensor speedups are measured with streaming TTFT on a single-request workload; throughput measurements at c=1–32 confirm zero degradation (9+ QPS, zero errors) on short prompts, and long-context throughput (8K XSum, c=1–8) achieves 14+ QPS with zero errors, confirming donor reuse at scale. (2) The quality model parameters are fit against published aggregate metrics rather than per-token quality measurements. Empirical output quality verification (Table 7) addresses this for the scenarios we evaluate. (3) SemBlend uses MiniLM-L6-v2 (384-dim) which has lower embedding quality than jina-v4 (1024-dim). An embedding ablation (Table 13 in §4.4) shows that MiniLM cosine accepts 5/7 variation types at τ=0.60 vs. Jaccard's 3/7; critically, MiniLM captures partial-80 and partial-60 types that Jaccard misses. For high-overlap variations (exact, reorder, paraphrase), both methods agree. In practice, 384-dim embeddings are sufficient for nearest-neighbor donor discovery in RAG workloads where the same

document is queried with variations. (4) Single-GPU experiments; multi-GPU tensor-parallel serving with sharded KV caches may change the overhead profile. (5) KV-tensor results use n=5–8 per prompt length with per-length server restarts, providing clean measurements but limiting total sample size per configuration.

Limitations. While RoPE delta correction (§3.6) solves position encoding exactly, **attention pattern drift** from context differences is addressed heuristically by the bathtub curve; a more principled per-request calibration could further improve quality. The bathtub presets are calibrated for Qwen2.5-7B and LLaMA-3.1-8B; GQA, MoE, and very deep models (>80 layers) may require recalibration. Cross-instance donor discovery via the optional CAGRA tier is validated on a controlled 8-worker fleet (§3.2), achieving 2.4–3.8× speedup on exact-match pairs; routing accuracy on diverse production workloads will depend on the semantic overlap distribution of incoming traffic. Empirical per-instance isolation is additionally confounded by vLLM's internal RadixAttention prefix cache. Short prompts (<500 tokens) may not benefit, as the ∼8 ms pipeline overhead can exceed the cold prefill time saved. **Benchmark scope.** Current evaluations use NLP summarization datasets (CNN/DM, XSum, WikiHow, SAMSum, MultiNews) and the WildChat conversational corpus. These cover important cases but are not purpose-built for semantic caching. Larger-scale, real-world datasets specifically designed to capture the distribution of semantically related but lexically distinct queries across diverse production workloads—code generation, customer support, search, multi-turn dialogue—are needed to more rigorously characterize the operational envelope of semantic KV reuse systems.

6 Conclusion

We presented SemBlend, a semantic KV cache system that extends LMCache's exact-prefix matching to semantic donor discovery via an **in-process architecture**. SemBlend performs donor discovery in ∼8 ms (MiniLM CPU embedding + numpy cosine + rapidfuzz alignment + calibrated bathtub) with zero network hops. For deployments exceeding 100K donors, the optional GPU-accelerated CAGRA tier provides cross-instance discovery at a measured 1.37 ms constant from N=100 to N=1M donors (A10G).

At the KV-tensor level, we solve the Rotary Position Embedding (RoPE) challenge that has constrained all prior semantic KV reuse systems to prefix-only scenarios. **Exact RoPE delta correction** (§3.6) exploits the rotation group structure to correct donor K tensors to arbitrary target positions at $O(Nd)$ cost per layer—mathematically exact, with zero approximation error. Combined with **calibrated bathtub recomputation** that addresses the orthogonal problem of attention pattern drift, SemBlend enables fully non-contiguous KV reuse (REORDER, PARTIAL, paraphrase) on all RoPE-based models.

Four takeaways stand out:

(1) Semantic KV reuse works at scale. SemBlend achieves **2.2**×–∼**12**× TTFT speedup across 2K–32K tokens on two model architectures, with TTFT remaining approximately constant (∼500 ms–1.3 s) regardless of prompt length. At 32K tokens cold prefill takes 15.4 s; SemBlend completes in 1.3 s. On five real NLP datasets, hit-only speedup ranges 1.97×–3.78× at 8K tokens.

(2) Quality is preserved. On datasets with ≥88% hit rate (WikiHow, XSum, synthetic), PPL ratio stays ≤1.065 on both models. Elevated PPL on lower-hit datasets (SAMSum, CNN/DM) is driven by miss-run outliers, not KV injection. The quality floor guarantee holds by construction: accepted hits use verified exact-match KV blocks; rejected queries fall back to cold prefill (PPL ≡ 1.0).

(3) Architecture scales from in-process numpy to GPU ANN. In-process numpy handles up to 10K donors at ≤0.14 ms; the optional cuVS brute-force tier covers up to 100K at ≤0.71 ms; CAGRA ANN achieves **1.37** ms *constant* at N=1M donors—in-process with no network hops on the critical path.

(4) Architecture-specific bathtub improves reuse quality. Selectively recomputing high-drift layers (LLaMA: early layers $[0, 1, 2]$ at 9.4%; Qwen: late layers $[26, 27]$ at 7.1%) addresses attention pattern drift orthogonal to RoPE position correction, consistent with architecture-specific layer criticality findings [26, 25].

SemBlend is, to our knowledge, the first system combining exact RoPE correction with cal-

ibrated layer-level recomputation to enable fully non-contiguous semantic KV reuse on production transformers.

Table 15: Triton kernel configurations for PartialAttention.

Kernel	**Grid**	**Block**	**Complexity**
scatter_donor_kv	$(P, 2LH)$	D	$O(P \cdot L \cdot H \cdot D)$
masked_qkv_proj	$(T, 1)$	M×K	$O(T \cdot 3HD \cdot D_m)$
partial_prefill	(C, H)	K	$O(C \cdot T \cdot D)$

P = copy pairs, L = layers, H = heads, D = head dim, T = seq len, C = compute positions, D_m = model dim.

A PartialAttention Kernel Specifications

The three Triton kernels use the following grid and block configurations:

B Reproduction

All code, datasets, and benchmark scripts are available in the project repository. The benchmark CLI accepts dataset selection, endpoint configuration, and output directory parameters. Results are stored as JSON with per-query metrics for independent analysis. Bootstrap confidence intervals use 10,000 resamples with the `scipy.stats.bootstrap` API.

References

[1] Y. Yao *et al.*, "CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion," in *Proc. EuroSys*, 2025.

[2] W. Zhang *et al.*, "KVShare: Semantic-Aware KV Cache Sharing for Multi-Tenant LLM Serving," arXiv:2503.16525, 2025.

[3] B. Li *et al.*, "SemShareKV: Semantic-Aware KV Cache Sharing with Token-Level LSH Matching," arXiv:2509.24832, 2025.

[4] S. Wang *et al.*, "ProphetKV: Dual-Stage KV Cache Optimization for RAG with User Query Relevance," arXiv:2602.02579, 2026.

[5] Y. Liu *et al.*, "LMCache: Distributed KV Cache Sharing for Large Language Model Serving," 2025.

[6] W. Kwon *et al.*, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proc. SOSP*, 2023.

[7] C. Ootomo *et al.*, "CAGRA: Highly Parallel Graph-Based Approximate Nearest Neighbor Search," in *Proc. ICDE*, 2024.

[8] T. Dao *et al.*, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in *Proc. NeurIPS*, 2022.

[9] R. Wagner and M. Fischer, "The String-to-String Correction Problem," *J. ACM*, 21(1):168–173, 1974.

[10] W. Kwon *et al.*, "vLLM: Efficient Memory Management for Large Language Model Serving," GitHub, 2023.

[11] Z. Bang *et al.*, "GPTCache: An Open-Source Semantic Cache for LLM Applications," GitHub, 2023.

[12] H. Chase, "LangChain: Building Applications with LLMs through Composability," GitHub, 2023.

[13] P. Tillet *et al.*, "Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations," in *Proc. MAPL*, 2019.

[14] W. Wang *et al.*, "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers," in *Proc. NeurIPS*, 2020.

[15] M. Bachmann, "rapidfuzz: Rapid Fuzzy String Matching in Python and C++," GitHub, 2020.

[16] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced Transformer with Rotary Position Embedding," *arXiv preprint arXiv:2104.09864*, 2021.

[17] K. Hermann *et al.*, "Teaching Machines to Read and Comprehend," in *Proc. NeurIPS*, 2015.

[18] S. Narayan, S. B. Cohen, and M. Lapata, "Don't Give Me the Details, Just the Summary! Topic-Aware Convolutional Neural Networks for Extreme Summarization," in *Proc. EMNLP*, 2018.

[19] A. Fabbri *et al.*, "Multi-News: A Large-Scale Multi-Document Summarization Dataset and Abstractive Hierarchical Model," in *Proc. ACL*, 2019.

[20] M. Koupaee and W. Y. Wang, "WikiHow: A Large Scale Text Summarization Dataset," *arXiv preprint arXiv:1810.09305*, 2018.

[21] B. Gliwa *et al.*, "SAMSum Corpus: A Human-annotated Dialogue Dataset for Abstractive Summarization," in *Proc. EMNLP Workshop*, 2019.

[22] W. Zhao *et al.*, "WildChat: 1M ChatGPT Interaction Logs in the Wild," in *Proc. ICLR*, 2024.

[23] NVIDIA, "NVIDIA Dynamo: A Datacenter-Scale Distributed Inference Serving Framework," `https://github.com/ai-dynamo/dynamo`, Feb. 2026.

[24] J. Yang, B. Hou, W. Wei, Y. Bao, and S. Chang, "KVLink: Accelerating Large Language Models via Efficient KV Cache Reuse," arXiv:2502.16002, Feb. 2025.

[25] Y. Liu *et al.*, "DroidSpeak: KV Cache Sharing for Cross-LLM Communication and Multi-LLM Serving," arXiv:2411.02820, Nov. 2024.

[26] S. Ananthanarayanan, A. Sengupta, and T. Chakraborty, "Understanding the Physics of Key-Value Cache Compression for LLMs through Attention Dynamics," arXiv:2603.01426, Mar. 2026.

[27] P. Poudel, "Stateful KV Cache Management for LLMs: Balancing Space, Time, Accuracy, and Positional Fidelity," arXiv:2511.04686, Nov. 2025.

[28] Z. Liu *et al.*, "KVCOMM: Online Cross-context KV-cache Communication for Efficient LLM-based Multi-agent Systems," in *Proc. NeurIPS*, 2025. arXiv:2510.12872.

[29] H. Yao *et al.*, "MEPIC: Memory Efficient Position Independent Caching for LLM Serving," arXiv:2512.16822, Dec. 2025.

[30] Y. Ding *et al.*, "EPIC: Efficient Position-Independent Context Caching for Serving Large Language Models," arXiv:2410.15332, Oct. 2024.

[31] X. Wang *et al.*, "SentenceKV: Efficient LLM Inference via Sentence-Level Semantic KV Caching," in *Proc. COLM*, 2025. arXiv:2504.00970.